MECHATRONIC PROJECT

SYSTEM DESIGN & PROJECT REPORT



SMART SPRAY SYSTEM

Remote Fire-Tracking & Autonmous Gyogssopic Turret

NAME	P.M.S.SANDARUWAN
MIS	IU/24/NDART2/2/0007
COURSE NAME	AUTOMATION & ROBOTICS
MODULE	MECHATRONIC INTEGRATED SYSTEM
MODULE CODE	D33T003M21
Semester	3RD SEMESTER

Contents

1 The Vision & Executive Summary **2**

2 Practical Applications (What is this project used for?) **2**

3 Core Objectives & System Metrics **3**

4 Deep Engineering Component Breakdown **4**

5 Operational Challenges (Problems Faced) **4**

6 Financial Investment & Budget Analysis **6**

7 Risk Assessment & Mitigation **8**

8 System Wiring & Circuit Architecture **9**

9 Receiver System Circuit Architecture (1.jpeg) **11**

10 Transmitter System Circuit Architecture (2.jpeg) **13**

A Core System Source Codes **15**

A.1 Transmitter Code (Helmet / Gyroscope).....15

A.2 Receiver Code (Turret State-Machine).....17

1. The Vision & Executive Summary

When a fire erupts in a critical industrial facility, every single second counts. However, the current standard for fire suppression is fundamentally flawed:

- **The Human Risk:** Sending personnel into toxic environments exceeding 1000°C to manually aim hoses often results in severe or fatal injuries.
- **The Efficiency Flaw:** Automated ceiling sprinklers spray indiscriminately. They frequently cause millions of rupees in collateral water damage to unaffected servers, electronics, and machinery.

The Mechatronic Solution

This report details a paradigm shift successfully developed for this project: **The Dual-Mode Smart Spray System**. Instead of blindly spraying water across an entire room, we deployed a highly targeted, mechatronic water turret.
In **Tele-operation Mode**, an operator seated in a safe bunker wears a gyroscopic helmet; the turret flawlessly mimics their head movements in real-time. In **AI-Override Mode**, an integrated computer vision system actively scans for the word “FIRE”. Upon detection, it overrides human input, locks onto the target autonomously, and delivers a surgical, high-pressure 12V water burst directly to the source.

2.Practical Applications (What is this project used for?)

This mechatronic system was designed to solve real-world industrial and safety challenges. Its primary applications include:

- **High-Value Asset Protection (Server Rooms):** Unlike ceiling sprinklers that flood entire rooms, this turret surgically extinguishes flames, saving unaffected servers and electronics from collateral water damage.

- **Hazardous Chemical Facilities:** It acts as a first responder in toxic or explosive environments where human entry is strictly prohibited or fatal.
- **Remote Industrial Wash-Down:** Beyond firefighting, the gyroscopic helmet allows operators to safely guide high-pressure water jets to clean chemical spills or nuclear waste from a safe bunker.

3.Core Objectives & System Metrics

To ensure this prototype functions as an industrial-grade unit rather than a basic toy, it was engineered to meet strict, measurable benchmarks:

1. **Zero-Lag Telemetry (<20ms Latency):** Standard Wi-Fi introduces a dangerous delay. In a crisis, half a second is the difference between life and death. By utilizing the ESP-NOW protocol, we bypass routers entirely, achieving a direct MAC-to-MAC connection. The machine feels like a natural extension of the operator's body.

2. **Autonomous Target Lock (Non-Blocking):** The computer vision system utilizes Optical Character Recognition (OCR) to detect the visual indicator ("FIRE"). This processing runs on a decentralized Python thread to ensure the live camera feed never freezes or drops frames.

3. **The "6-10" Suppression Loop:** Once triggered, the system independently executes a precise state machine: a 0.5-second warning buzzer, followed by a 6-second 12V high-pressure spray, concluding with a mandatory 10-second hardware cooldown to prevent motor burnout and conserve water.

1. Deep Engineering Component Breakdown

Every component in this system was carefully selected to solve specific mechanical or electrical challenges inherent in fluid robotics.

Component Used	Engineering Justification
ESP32 S3 N16R8	**Dual-Core Brain:** Standard microcontrollers crash when handling Wi-Fi and Servo calculations simultaneously. The S3's dual cores easily manage ESP-NOW packets while generating perfectly smooth PWM signals.
MG996R Metal Servos	**High-Torque Actuation:** Standard plastic servos strip their gears under the weight of a water-filled nozzle. The MG996R provides 11kg/cm of torque, ensuring stable, jitter-free aiming.
IRLZ44N MOSFETS	**Power Isolation:** A 3.3V logic pin will fail if connected directly to a 12V pump. These MOSFETs act as heavy-duty electronic gates, allowing a tiny 3.3V signal to safely switch high-amperage 12V loads.
10,000µF Capacitors	**Brownout Protection:** When four servos start moving simultaneously, they draw massive current, which can reboot the system. These capacitors store energy and release it instantly to smooth out inductive spikes.

2. Operational Challenges (Problems Faced)

During the construction and live-fire testing of this prototype, several physical and software challenges emerged that required immediate engineering solutions:

- **Payload Inertia & Momentum:** The physical weight of the water rushing through the nozzle created unexpected momentum. Standard

plastic servos could not hold their position and caused the aim to "sag". This forced a mid-project upgrade to high-torque metal-gear servos.

- **Microcontroller Brownouts:** Operating the water pump and the pan-tilt servos simultaneously drew excessive current, causing the main ESP32 processor to restart randomly. This was resolved by isolating the logic power from the motor power using dedicated buck converters and capacitors.
- **Telemetry Latency:** Initial testing over standard Wi-Fi introduced a half-second delay. When the operator moved their head, the turret lagged behind, causing severe disorientation. Switching to the MAC-to-MAC ESP-NOW protocol entirely eliminated this latency.

3. Financial Investment & Budget Analysis

Budgeting for a mechatronic project requires balancing cost with reliability. Below is a detailed breakdown comparing the initial estimated budget against the actual engineering expenditure. Every item is listed individually for absolute transparency.

Initial Estimated Budget

The original budget was calculated for a basic, low-power prototype.

Component Description	Estimated Cost (LKR)
ESP32 Development Boards	4,500
Camera Module	3,000
Motion Sensor Module	1,000
Manual Control Joysticks	1,000
Standard Servo Motors	5,500
Pan-Tilt Bracket	3,000
Mini DC Water Pump	2,000
Water Tank & Tubing	2,500
Relay Module	1,500
Switches & Push Buttons	1,000
Power Supply Unit	4,000
Wires & Connectors	2,000
Mounting Hardware	1,000
Miscellaneous Consumables	2,000
Estimated Total	**34,000**

Actual Engineering Expenditure

During feasibility testing, it was determined that standard plastic servos and direct-drive logic pins would fail under the weight of the water payload. To guarantee industrial-grade reliability, critical structural and electrical upgrades were implemented.

Upgraded Component Listing	Qty	Cost (LKR)
ESP32 S3 N16R8 Microcontrollers	4	6,760
ESP32 CAM RHYX/0V2640 Modules	2	4,780
MG996R Metal Gear Servos	4	5,000
Pan-Tilt Disks & Brackets	-	970
12V High-Pressure Water Pump	1	900
5V Mini Water Pump	1	480
18650 Li-ion Batteries	6	2,500
Dual Battery Charger	1	640
12V/5A Power Supply Unit	2	1,000
LM2596 Buck Converters	4	800
L298N Motor Drivers	2	800
IRLZ44N MOSFETs	10	600
10000μF Capacitors	16	600
MPU-6050 Gyroscope	2	800
BMI-160 Inertial Measurement Unit	2	1,000
PIR Motion Sensors	2	650
OLED Display Modules	2	1,500
Analog Joystick Modules	4	1,000
Vero Boards	-	2,000
Project Enclosures	-	2,520
Wiring, Terminals & Hardware	-	2,000
Total Capital Utilized		**37,300**

This strategic engineering decision brought the actual final expenditure to 37,300 LKR, ensuring a 100% stable, resilient, and robust system.

4. Risk Assessment & Mitigation

To guarantee the system operates safely and effectively in a crisis, a comprehensive risk assessment was conducted. Engineering safeguards were implemented to mitigate each identified threat.

Identified Risk	Potential Impact	Mitigation Strategy
Hardware Inductive Kickback	Reverse voltage from the deactivating pump destroys the ESP32.	Implemented Flyback Diodes and optically isolated relays to protect delicate logic circuits.
UI Freezing & Video Stutter	OCR processing blocks the live camera feed, blinding the operator.	Deployed Python Multi-threading to completely separate video rendering from AI processing.
False AI Positives	Turret autonomously fires at non-threat text, wasting water.	Tuned OCR confidence thresholds and required 3 consecutive frames of detection before firing.
Water Leakage & Shorts	High-pressure water leaks onto the turret's exposed electronics.	Sealed all vital connections with heat-shrink tubing and housed controllers in waterproof enclosures.

5. System Wiring & Circuit Architecture

To ensure successful replication and guarantee electrical safety, the hardware is strictly divided into two isolated circuits communicating wirelessly via the ESP-NOW protocol. The following tables strictly adhere to the verified component connection summaries.

5.1 Transmitter Circuit (Helmet Gyro & Vision)

This circuit captures the operator's head movements and processes the AI vision feed. It is powered via a 12V supply stepped down to 5V, providing clean logic power for the sensors.

Component	Module Pin	ESP32-S3 Connection
MPU-6050 Gyroscope	SDA (Data Line)	GPIO 8 (I2C SDA)
MPU-6050 Gyroscope	SCL (Clock Line)	GPIO 9 (I2C SCL)
ESP32-CAM (Vision)	IO14 (TX Serial Pin)	GPIO 17 (UART RX)
ESP32-CAM (Vision)	IO13 (RX Serial Pin)	GPIO 18 (UART TX)
Sensor Power	VCC (MPU) / 5V (CAM)	3V3 Pin / 5V Pin

5.2 Receiver Circuit (Turret, Pump & Manual Control)

This mixed-signal circuit manages the high-torque actuation, the water pump, and the manual operator override controls (joysticks and buttons). It utilizes dedicated buck converters to strictly isolate logic power from motor power.

Critical Wiring Note: *According to the true connection summary, ensure all component grounds are tied to the* ***COMMON GND****. The servos must receive isolated* ***6V from Buck 2****, the ESP32 receives* ***5V from Buck 1****, and the pump receives* ***12V Direct*** *from the power supply.*

Component	Circuit Routing	ESP32-S3 Pin
MG996R Pan Servo	Pan Servo Signal	GPIO 16 (PWM)
MG996R Tilt Servo	Tilt Servo Signal	GPIO 17 (PWM)
Left/Right Joystick	X-Axis Signal	GPIO 6 (Analog)
Up/Down Joystick	Y-Axis Signal	GPIO 15 (Analog)
Manual Override	High Spray Button Signal	GPIO 18 (Digital)
Manual Override	Low Spray Button Signal	GPIO 8 (Digital)
Warning Buzzer	Buzer (+) Terminal	GPIO 4 (Digital Out)
IRLZ44N MOSFET	Gate Pin (via 10kΩ Resistor)	GPIO 21 (Digital Out)

Receiver System Circuit Architecture ()

Turret Base, Manual Overrides & Power Distribution

This system manages the high-torque pan-tilt actuation, MOSFET-driven water pump, and analog manual overrides. Wiring below is verified against the official connection summary.

Component Name	Wire / Signal Function	ESP32-S3 Connection
Pan Servo (MG996R)	Signal Wire	GPIO 16
Tilt Servo (MG996R)	Signal Wire	GPIO 17
Left / Right Joystick	X-Axis Signal	GPIO 6
Up / Down Joystick		
High Spray Button	Signal Output	GPIO 18
Low Spray Button	Signal Output	GPIO 8
Buzzer Module	Positive (+) Terminal	GPIO 4
IRLZ44N MOSFET	Gate (via 10kΩ pull-down)	GPIO 21
System Ground	Common Ground Tie	All GND Pins

Power Distribution Summary		
ESP32-S3 Power	5V Input	5V From Buck Converter 1
Servo Motor Power	VCC (+)	6V From Buck Converter 2
Mini Water Pump	Positive (+)	12V Direct from Power Supply
Joystick Power	VCC (+)	3.3V From ESP32-S3 Pin

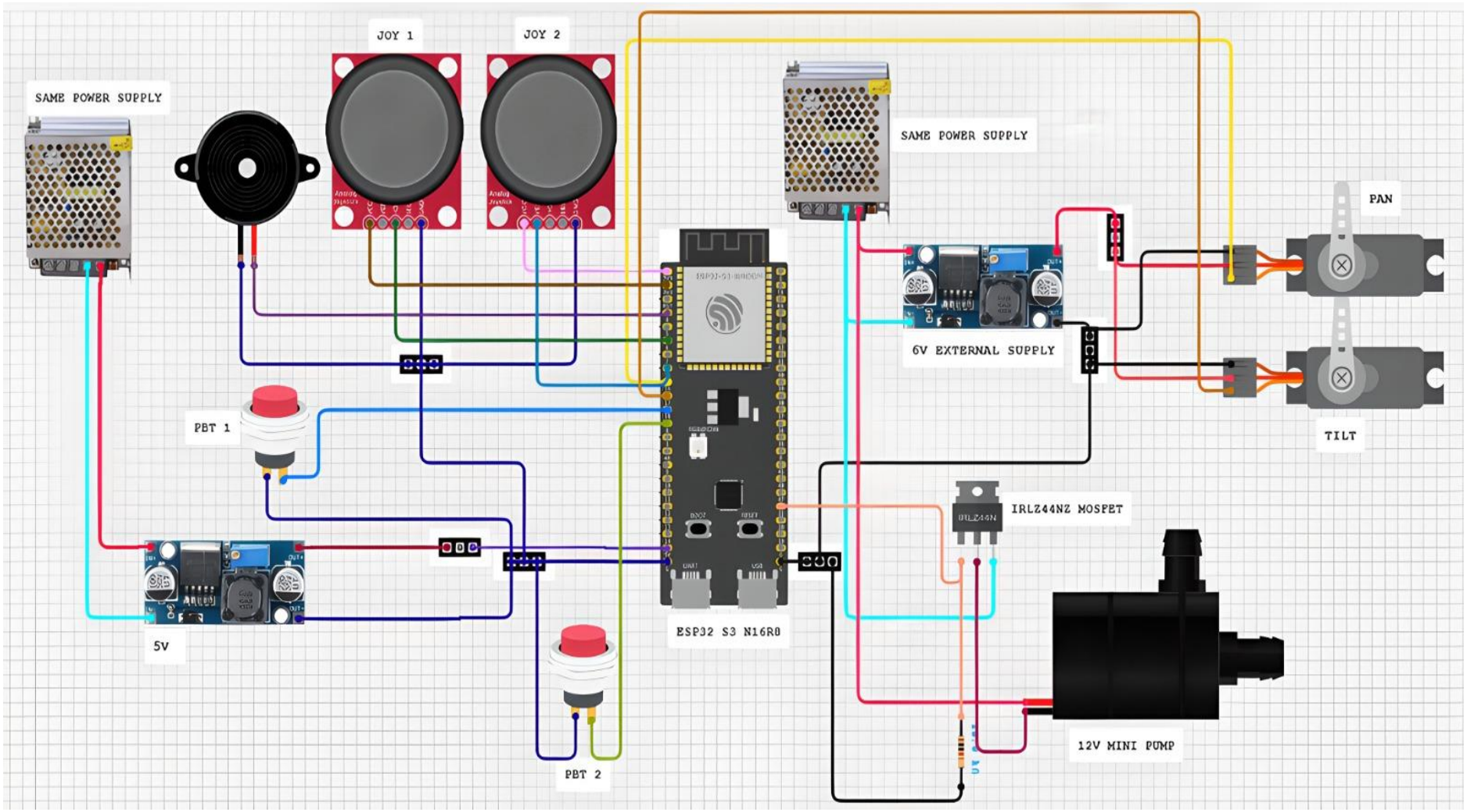


Figure 1: Receiver System Circuit Architecture (Verified)

10. Transmitter System Circuit Architecture (2.jpeg)

Helmet Gyroscope & AI Vision Module

This highly sensitive logic circuit manages real-time spatial orientation data via I2C and Serial UART communication. Wiring is verified against the official connection summary.

Component Name	Wire / Signal Function	ESP32-S3 Connection
ESP32-CAM Module	IO14 (TX Serial Pin)	GPIO 17 (RX Pin)
ESP32-CAM Module	IO13 (RX Serial Pin)	GPIO 18 (TX Pin)
MPU-6050 Gyroscope	SDA (I2C Data Line)	GPIO 8
MPU-6050 Gyroscope	SCL (I2C Clock Line)	GPIO 9
System Ground	Common Ground Tie	All GND Pins
	Power Distribution Summary	
ESP32-S3 Power	5V Input	5V From LM2596 Step-Down
ESP32-CAM Power	5V Input	5V Pin on ESP32-S3
MPU-6050 Power	VCC (+)	3.3V Pin on ESP32-S3
Main Power Input	LM2596 Input	12V Direct from Power Supply

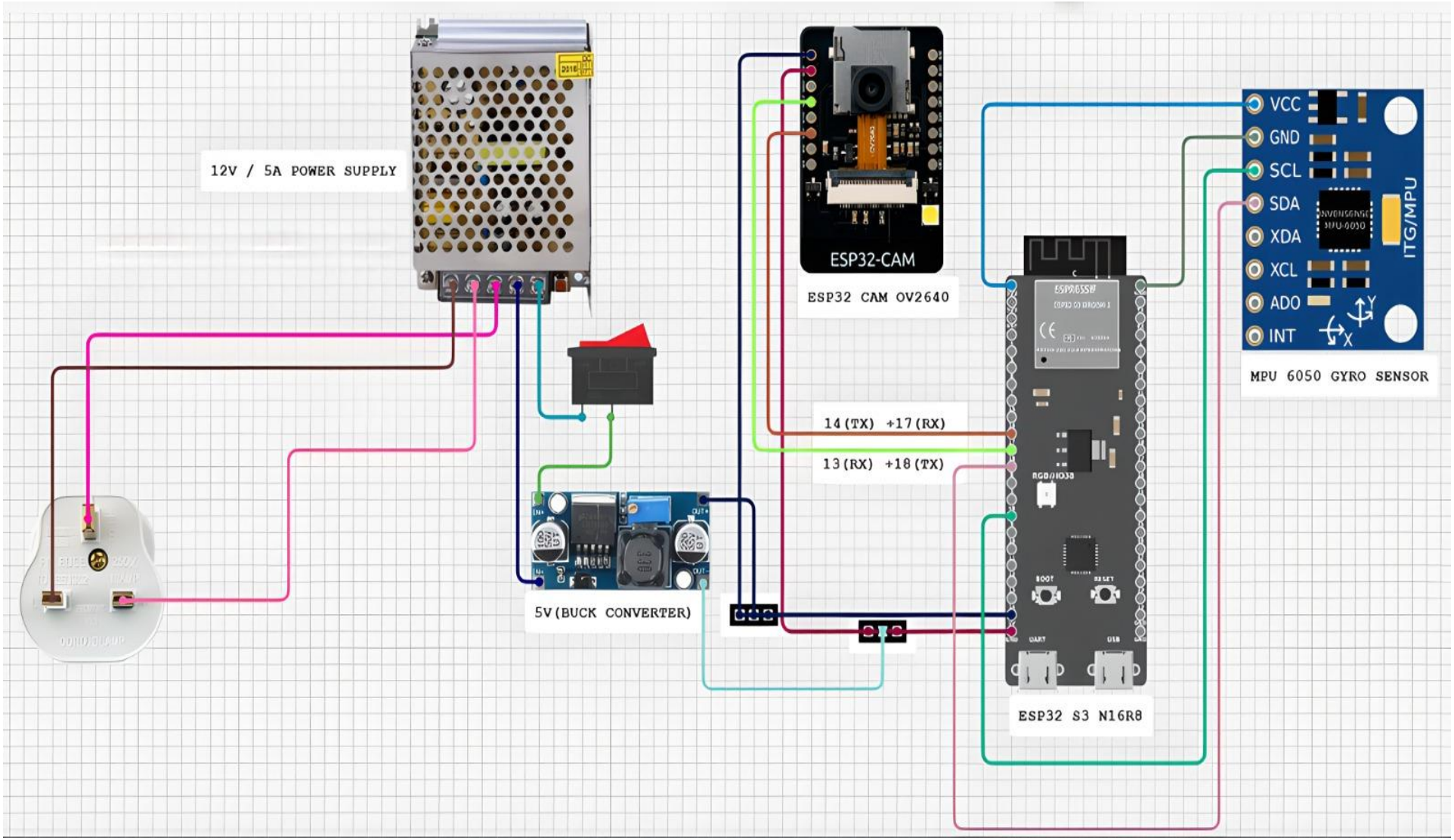


Figure 2: Transmitter System Circuit Architecture (Verified)

A. Core System Source Codes

A.1. Transmitter Code (Helmet / Gyroscope)

This code reads the MPU-6050 sensor data on the operator's helmet and securely transmits the physical coordinates via ESP-NOW.

```
#include  <Wire.h>
#include  <Adafruit_MPU6050.h>
#include  <esp_now.h>
#include   <WiFi.h>

Adafruit_MPU6050 mpu;
uint8_t rxAddress[] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};

// Custom data payload structure
typedef struct ControlPacket {
    uint32_t fireEventId;
    float pan, tilt, fireOffsetX, fireOffsetY;
    uint8_t flags; // 0x01 triggers Fire Override mode
} ControlPacket;

ControlPacket sendData;
esp_now_peer_info_t peerInfo;

void setup() {
    Wire.begin();
    mpu.begin();
    mpu.setAccelerometerRange( MPU6050_RANGE_8_G);

    WiFi.mode(WIFI_STA);
    esp_now_init();

    //   Register   the   receiver
    memcpy(peerInfo.peer_addr, rxAddress, 6);
    peerInfo.channel = 0;
    peerInfo.encrypt = false;
    esp_now_add_peer(&peerInfo);
}

void loop() {
```

```
    sensors_event_t a, g, temp;
    mpu.getEvent(&a, &g, &temp);

    // Map acceleration geometry to 180-degree servo range neatly
    sendData.pan =constrain(map(a.acceleration.x*100,-1000,1000,0,180),0,180
    sendData.tilt =constrain(map(a.acceleration.y*100,-1000,1000,0,180),0,18
    sendData.flags = 0x00; // Normal Operator Mode

    esp_now_send(rxAddress, (uint8_t *) &sendData, sizeof(sendData));

    // Transmit at approximately 60Hz for zero-latency feel
    delay(15);
}
```

A.2. Receiver Code (Turret State-Machine)

This script receives the telemetry, manages the high-torque servo actuation, and executes the automated fire suppression sequence.

```
#include <esp_now.h>
#include <WiFi.h>
#include <ESP32Servo.h>

Servo panServo, tiltServo;
const int panPin = 16, tiltPin = 17, pumpPin = 21, buzzerPin = 4;

typedef struct ControlPacket {
    uint32_t fireEventId;
    float pan, tilt, fireOffsetX, fireOffsetY;
    uint8_t flags;
} ControlPacket;

ControlPacket rxData;
enum SystemMode { GYRO_MODE, FIRE_LOCK_MODE };
SystemMode currentMode = GYRO_MODE;

enum FireState { IDLE, BUZZER_ON, PUMP_ON, COOLDOWN };
FireState fireState = IDLE;
unsigned long fireTimer = 0;
float locked_yaw = 90.0, locked_pitch = 90.0;

void OnDataRecv(const uint8_t *mac_addr, const uint8_t *inData,int len){
    memcpy(&rxData, inData, sizeof(rxData));

    // AI Override Triggered by Camera
    if ((rxData.flags & 0x01) && currentMode != FIRE_LOCK_MODE) {
        locked_yaw = constrain(rxData.pan - rxData.fireOffsetX, 0, 180);
        locked_pitch =constrain(rxData.tilt -rxData.fireOffsetY,0,180);
        currentMode = FIRE_LOCK_MODE;
        fireState = IDLE;
    }
}

void setup() {
    panServo.attach(panPin);
    tiltServo.attach(tiltPin);
```

```
    pinMode(pumpPin, OUTPUT);
    pinMode(buzzerPin, OUTPUT);

    WiFi.mode(WIFI_STA);
    esp_now_init();
    esp_now_register_recv_cb(OnDataRecv);
}


void loop() {
    unsigned long now = millis();

    if (currentMode == FIRE_LOCK_MODE) {
        panServo.write(locked_yaw);
        tiltServo.write(locked_pitch);

        // Sequence: Buzzer (0.5s) -> Spray (6s) -> Cooldown (10s)
        if (fireState == IDLE) {
            digitalWrite(buzzerPin, HIGH);
            fireState = BUZZER_ON;
            fireTimer = now;
        } else if (fireState == BUZZER_ON && (now - fireTimer >= 500)) {
            digitalWrite(buzzerPin, LOW);
            digitalWrite(pumpPin, HIGH); // Open MOSFET Gate
            fireState = PUMP_ON;
            fireTimer = now;
        } else if (fireState == PUMP_ON && (now - fireTimer >= 6000)) {
            digitalWrite(pumpPin, LOW);
            fireState = COOLDOWN;
            fireTimer = now;
        } else if (fireState == COOLDOWN && (now - fireTimer >= 10000)){
            currentMode = GYRO_MODE;
            fireState = IDLE;
        }
    } else {
        // Normal Operator Telemetry Mode
        panServo.write(constrain(rxData.pan, 0, 180));
        tiltServo.write(constrain(rxData.tilt, 0, 180));
    }
    delay(15);
}
```

Project Development Timeline

The following 9-week schedule details the end-to-end development phases of the Smart Spray System, from the initial feasibility study to final system delivery and documentation.

Week	Date	Phase Description
Week 1	Apr 08 – Apr 14	**Feasibility Study:** Analyzed system requirements and conceptualized the design.
Week 2	Apr 15 – Apr 21	**Procurement:** Acquired initial parts. Discovered missing components, requiring a second trip.
Week 3	Apr 22 – Apr 28	**Prototyping & Troubleshooting:** Breadboard testing exposed electrical shorts. Redesigned circuit.
Week 4	Apr 29 – May 05	**Assembly & Soldering:** Transitioned to Vero boards. Overcame component damage during soldering.
Week 5	May 06 – May 12	**Telemetry Firmware:** Programmed C++ code. Captured MPU-6050 data and established ESP-NOW link.
Week 6	May 13 – May 19	**Turret Actuation:** Programmed receiver state machine, calibrated PWM servos, and validated MOSFET logic.
Week 7	May 20 – May 26	**AI Integration:** Configured OpenCV pipeline for OCR threat marker detection using the ESP32-CAM.
Week 8	May 27 – Jun 02	**System Calibration:** Merged hardware and AI. Conducted live-fire testing of the suppression loop.
Week 9	Jun 03 – Jun 05	**Final Delivery:** Compiled data, finalized diagrams, and formatted the official engineering report.